Facultad de Ingeniería en Ciencias de la Computación y Telecomunicaciones,
UniversidadAutónoma Gabriel René Moreno

ESTUDIO DE MERCADO

PROYECTO: Sistema Distribuido de Gestión Empresarial y Liquidación de Siniestros de Seguros
Materia: **Ingeniería de Software II**

N.º	Integrantes	Registro
1	Garcia Caballero David	217058795
2	Garcia Caballero Jose Diego	221044523
3	Velasquez Paz Arnulfo	220030383

Santa Cruz-Bolivia

1.2. Cronograma de 4 Semanas de Desarrollo....3
3. Arquitectura del Sistema en 4 Niveles....4
3.1. Nivel 1: Frontend Heterogéneo (3 Tecnologías)....4
3.2. Nivel 2: Dominio ERP y Microservicios (Mínimo 3 Lenguajes)....4
3.3. Nivel 3: Persistencia Políglota y Agregación (3 Motores de BD)....4
3.4. Nivel 4: Arquitectura General e Infraestructura....5
4. Patrones de Código, Transacciones Distribuidas y Automatización....5
4.1. Tres Patrones de Arquitectura a Nivel de Código....5
4.2. Transacciones Distribuidas: Patrón Saga Orquestado e Idempotencia....5
4.3. Automatización de Flujos Multicanal con n8n....6
5. Inteligencia Artificial, Reportes por Voz y Seguridad....6
5.1. Protocolo MCP y Agentes Autónomos de Monitoreo....6
5.2. Machine Learning Supervisado, No Supervisado y Edge AI....6
5.3. Generación de Reportes mediante Voz y Lenguaje Natural....6
5.4. Seguridad Integral y Prevención OWASP....7

El diseño y desarrollo sigue el proceso ágil y trazable, reduciendo la brecha entre el análisis y el código mediante 4 fases formales:

1. **Requisitos y Casos de Uso:** Modelado del dominio asegurador, redacción formal de casos de uso (RF01-RF07) y prototipado rápido de interfaces.
2. **Análisis de Robustez:** Conexión de casos de uso con objetos del sistema clasificados en:
    - **Boundary (Frontera):** Vistas en Angular, React, Flutter y capturadores de voz/audio.
    - **Control (Lógica):** Controladores de caso de uso en NestJS, motor actuarial en Spring Boot y agentes IA en Python.
    - **Entity (Persistencia):** Modelos de datos en PostgreSQL, MongoDB y DynamoDB.
3. **Diseño Detallado:** Diagramas de secuencia y definición de contratos de interfaces (GraphQL Schemas, contratos MCP y eventos Kafka).
4. **Implementación y Pruebas:** Codificación guiada por patrones y pruebas de integración distribuida (Saga y compensaciones).

1.2. Cronograma de 4 Semanas de Desarrollo

- **Semana 1:** Casos de uso formales, diseño de diagramas de robustez y especificación de contratos de datos (GraphQL/MCP).
- **Semana 2:** Microservicios base (Python, NestJS, Spring Boot, PHP), esquemas políglotas (PostgreSQL, MongoDB, DynamoDB) y orquestación Saga.
- **Semana 3:** Integración de los 3 frontends (Angular, React, Flutter), agentes autónomos con skills, flujos n8n y modelos de ML.
- **Semana 4:** Soporte offline con idempotencia, reportes generativos por voz a Excel, blindaje OWASP y pruebas de concurrencia.
- **Criterio de Evaluación:** Calidad, coherencia y corrección del diseño arquitectónico por encima del volumen de código.

2. Requisitos Funcionales Formales (RF01 - RF07)

RF01 - Gestión Integral del Dominio ERP: Registro y seguimiento integral de pólizas, asegurados, coberturas, radicación de siniestros, tasación pericial, liquidación actuarial, cuentas por pagar a talleres y contabilidad.

RF02 - Agentes Autónomos de Monitoreo: Supervisión continua del negocio mediante agentes basados en **Metas y Habilidades (Goals & Skills)** para alertar cuellos de botella, riesgos de liquidez o fraudes sin intervención humana directa.

RF03 - Operatividad Offline e Idempotencia: Funcionamiento continuo de las apps móvil y web sin conexión de red; sincronización garantizada mediante llaves universales (Idempotency-Key en DynamoDB) que evitan duplicidades tras reconexión.

RF04 - Automatización de Flujos End-to-End: Orquestación de procesos multicanal en **n8n** donde la salida de un servicio dispara el siguiente (Recepción de fotos por WhatsApp

Peritaje IA → Core ERP → Factura → Despacho).

RF05 - Reportes por Voz y Lenguaje Natural: Consultas analíticas mediante notas de voz o texto libre (Whisper STT + LLM), generando gráficos en pantalla y exportación inmediata a planillas **Microsoft Excel** (.xlsx) y PDF.

RF06 - Acceso Unificado (Single Sign-On - SSO): Autenticación centralizada con **Keycloak (OIDC / OAuth 2.0 + PKCE)**: una única sesión activa para navegar transparentemente por todos los módulos web y móviles.

RF07 - Transacciones Distribuidas Transversales: Coordinación de acciones complejas en microservicios bajo el **Patrón Saga Orquestado** con compensaciones semánticas ante fallos de red o caídas de servicios.

3. Arquitectura del Sistema en 4 Niveles

3.1. Nivel 1: Frontend Heterogéneo (3 Tecnologías)

1. **Angular (Web Principal):** Panel administrativo, formulación reactiva compleja para tasadores, auditores y directores; tipado TypeScript estricto y control RBAC.
2. **React (Web Paralelo):** Portal ligero de autoservicio para asegurados y corredores; alta reactividad para radicación ágil de siniestros vía Apollo Client.
3. **Flutter / PWA (Cliente Móvil):** Aplicación para peritos de campo en terreno. Soporta almacenamiento local (SQLite/Drift), sincronización diferida y ejecución de IA local (Edge AI).

- **Shell Host Container:** Orquesta rutas compartidas, inyecta el token Bearer JWT de Keycloak y mantiene el bus de eventos en memoria.

3.2. Nivel 2: Dominio ERP y Microservicios (Mínimo 3 Lenguajes)

- **Java (Spring Boot 3 - Lenguaje Clásico):** Cálculos financieros y actuariales de alta precisión, depreciaciones y reservas matemáticas.
- **TypeScript (NestJS 10 - Node.js):** Gestión del ciclo de vida del siniestro, ingesta de evidencias, I/O asíncrono no bloqueante y WebSockets.
- **Python (FastAPI + LangGraph - Obligatorio IA):** Agentes de monitoreo, servidor MCP y ejecución de modelos de Machine Learning.
- **PHP 8.3 (Laravel con Swoole - Lenguaje Clásico):** Gestión de pólizas, emisión de facturas fiscales y órdenes de pago bancarias.

3.3. Nivel 3: Persistencia Políglota y Agregación (3 Motores de BD)

- **PostgreSQL (SQL Relacional):** Integridad referencial estricta y transacciones ACID para la verdad contable y financiera.
- **MongoDB (NoSQL Documental):** Estructura flexible para expedientes de siniestros heterogéneos y metadatos periciales.
- **Amazon DynamoDB (NoSQL Clave-Valor Cloud):** Acceso sub-milisegundo para llaves de idempotencia, caché de pólizas activas y control de concurrencia.
- **Agregación y Caching:** Clúster de **Redis** para métricas calientes, réplicas de lectura y proyecciones CQRS que consolidan reportes sin bloqueos en la base de datos transaccional.

3.4. Nivel 4: Arquitectura General e Infraestructura

- **Kubernetes (EKS/GKE Multi-AZ) + Istio:** Malla de servicios con mTLS obligatorio y Network Security Policies (NSP) Zero Trust.
- **Apollo Supergraph Router:** Gateway GraphQL federado que unifica los subgrafos de los microservicios sin sobre-fetching.
- **Apache Kafka:** Broker distribuido para mensajería asíncrona, eventos de dominio y orquestación de compensaciones Saga.

4. Patrones de Código, Transacciones Distribuidas y Automatización

4.1. Tres Patrones de Arquitectura a Nivel de Código

Arquitectura por Capas (Layered): En el microservicio de Pólizas y Facturación (**PHP**). Separación jerárquica clara: Presentación

Lógica de Negocio → Acceso a Datos → Persistencia DynamoDB.

Arquitectura Hexagonal (Ports and Adapters): En el motor de Liquidación Actuarial (**Java Spring Boot**). Núcleo actuarial puro aislado de tecnologías externas; puertos de entrada/salida implementados por adaptadores GraphQL y JPA.

Clean Architecture + CQRS: En el microservicio de Reclamos y Triage (**NestJS**). Regla de dependencias concéntricas hacia entidades de dominio; división entre comandos de escritura en MongoDB y consultas optimizadas.

4.2. Transacciones Distribuidas: Patrón Saga Orquestado e Idempotencia

Al coordinar procesos que impactan a 3 o más microservicios (ej. Liquidar siniestro

Pólizas + Liquidación + Facturación):

- **Saga Orquestada:** Un orquestador central coordina los pasos en orden lógico:
- 1. Bloquear Cobertura (PHP)⟶2. Reservar Fondos (Spring Boot)⟶3. Emitir Factura (NestJS)

- 1. Bloquear Cobertura (PHP)⟶2. Reservar Fondos (Spring Boot)⟶3. Emitir Factura (NestJS)

Compensación Semántica (Rollback): Si el paso 3 falla por timeout o rechazo, el orquestador dispara acciones inversas: libera los fondos reservados en Spring Boot y desbloquea la cobertura en PHP, garantizando consistencia eventual.

Mecanismo de Idempotencia: El cliente genera un Idempotency-Key (

- UUIDv4+Hash del payload). DynamoDB verifica la existencia de la llave: si ya fue procesada, retorna el resultado previo sin reejecutar cobros ni duplicar asientos contables.

4.3. Automatización de Flujos Multicanal con n8n

5. Inteligencia Artificial, Reportes por Voz y Seguridad

5.1. Protocolo MCP y Agentes Autónomos de Monitoreo

- **Protocolo MCP (Model Context Protocol):** Expone herramientas seguras (consultar_poliza, evaluar_baremo, auditar_reserva) y recursos de contexto para que los LLMs interactúen con el ERP de forma estandarizada.
- **Agentes Basados en Metas y Skills:** Monitorean continuamente el Event Bus de Kafka sin intervención humana para resolver metas:
    - *Meta:* Detectar siniestros graves detenidos
    - *Skill:* Reasignar perito automáticamente por geocerca.
    - *Meta:* Proteger reservas técnicas. *Skill:* Emitir alertas de liquidez y desvío actuarial a directores.

5.2. Machine Learning Supervisado, No Supervisado y Edge AI

- **Modelos Supervisados:**
    - Visión Computacional: **YOLOv8** (detección de partes dañadas) y **Mask R-CNN** (segmentación de abolladuras y cálculo de área).
    - Audio: **Whisper STT** supervisado para transcribir comandos periciales por voz.
- **Modelos No Supervisados:**
    - Detección de Anomalías: **Isolation Forest** y **Deep Autoencoders** para identificar siniestros fraudulentos sin patrones previos.
    - Clustering: **HDBSCAN / K-Means** para agrupar redes de colusión entre talleres, peritos y asegurados recurrentes.
- **IA Local Embebida (Edge AI):**
    - **ONNX Runtime Web** (Wasm/WebGPU) en navegadores para validar nitidez de fotos en origen.
    - **LiteRT / TensorFlow Lite** en Flutter para extracción OCR de licencias sin internet.

5.3. Generación de Reportes mediante Voz y Lenguaje Natural

1. El usuario graba un audio: *"Genera un reporte de siniestros con liquidaciones mayores a $10.000 del último trimestre"*.
2. **Whisper** transcribe el audio
3. →
4. → Agente con **MCP** deduce dimensiones y genera la consulta analítica a las Read Replicas.
5. Se proyectan gráficos dinámicos en la UI y se genera automáticamente una planilla **Microsoft Excel** (.xlsx) con fórmulas y estilos, y un PDF ejecutivo para descarga inmediata.

5.4. Seguridad Integral y Prevención OWASP

- **Prevención de Inyección SQL (SQLi):** Consultas parametrizadas obligatorias vía ORMs tipados (Spring Data JPA y PDO sin emulación en PHP); validación estricta de DTOs en el perímetro (class-validator y Bean Validation @Valid).
- **Blindaje contra Prompt Injection:** Implementación de **NeMo Guardrails**, sandboxing de prompts y **JSON Schema Enforcement** estricto: el LLM nunca ejecuta consultas arbitrarias, solo parametriza llamadas a herramientas MCP validadas.
- **Single Sign-On (SSO):** Servidor central **Keycloak** con **OAuth 2.0 / OpenID Connect + PKCE**. Los tokens JWT (RS256) se propagan en memoria entre microfrontends sin reautenticación.
- **Zero Trust:** Service Mesh Istio con cifrado bidireccional mTLS y aislamiento estricto por Network Policies (NSP).